Simple numerical operations with vectors and matrices

Michał Sikora

October 2026

1 Method:

For this project, the C++ programming language (C++17 standard) was utilized to implement and analyze classical matrix and vector operations. A key concept in efficient numerical computing is proper memory management. Thus, two-dimensional matrices of size $n \times n$ (where $n = LD$) were stored in the computer memory as continuous one-dimensional arrays using `std::vector`. In order to preserve bad memory management and ensure seamless integration with external numerical libraries, the matrices were arranged in a row-major order. Accessing a specific element at the i-th row and j-th column was achieved via the index transformation:

$$index = i \cdot LD + j \tag{1}$$

where LD stands for the Leading Dimension of a specific matrix.

The core algorithm iteratively performed $m = 30$ matrix-vector multiplications. $\vec{x}^{(m)} = A^m \vec{x}$. A single matrix-vector multiplication $\vec{y} = A\vec{x}$ was realized using two nested loops, following the equation:

$$y_i = \sum_{j=0}^{n-1} A_{i,j} x_j \tag{2}$$

After each multiplication step, the Euclidean norm of the resulting vector was calculated according to the formula

$$||\vec{x}||_2 = \sqrt{\vec{x}^T \vec{x}} = \sqrt{\sum_{i=0}^{n-1} x_i^2} \tag{3}$$

The final stage of the project involved comparing a custom, naive implementation of matrix-matrix multiplication ($C = A \cdot A$) with an optimized external `cblas` library. The custom approach relied on the standard mathematical definition:

$$C_{i,k} = \sum_{j=0}^{n-1} A_{i,j} \cdot B_{j,k} \tag{4}$$

which requires three nested loops and exhibits a computational complexity of $O(n^3)$.

2 Tasks:

The practical part of the project required implementing the numerical operations described in the method section for a specific matrix and observing the behavior of the results.

First, a square matrix A of size $n = 2000$ (with leading dimension $LD = 2000$) was generated. Its elements were populated according to the following mathematical formula:

$$A_{i,j} = \frac{1}{|i-j|^4 + \gamma} \quad \text{for} \quad i, j = 0, 1, 2, \ldots, n-1 \tag{5}$$

The operations were conducted for two independent cases using different values of the parameter:

$$\gamma = 2 \text{ and } \gamma = 3.$$

The first task focused on repeated matrix-vector multiplication. A starting vector $\vec{x}^{(0)}$ of size $n = 2000$ was initialized entirely with ones ($x_i = 1$ for all i). The program then executed $m = 30$ consecutive multiplications, where the output vector from the previous step became the input for the next ($\vec{x}^{(i)} = A\vec{x}^{(i-1)}$). After each iteration, the Euclidean norm of the current vector was calculated and saved to a (CSV) file.

The second task aimed to benchmark the matrix-matrix multiplication $C = A \cdot A$. The execution time was measured for two different approaches:

- A custom, naive implementation using nested loops and 1D row-major array indexing.
- The highly optimized `cblas_dgemm` routine from the CBLAS library.

3 Results:

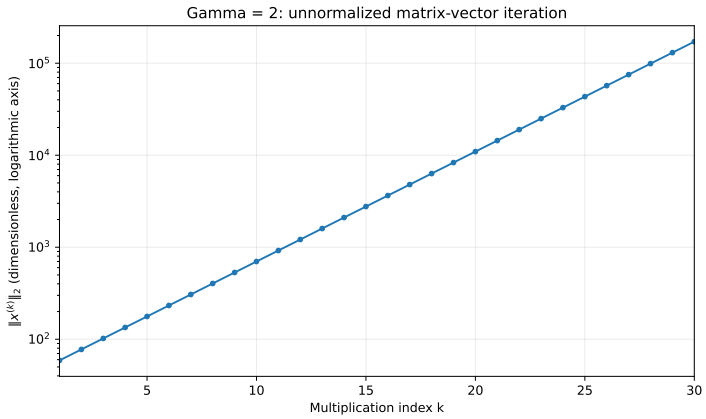


Figure 1: Change of the vector norm for $\gamma = 2$

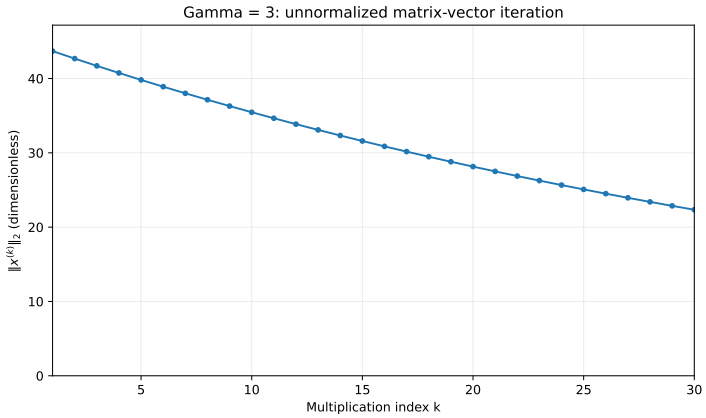


Figure 2: Change of the vector norm for $\gamma = 3$

Table 1: Comparison of execution times for matrix-matrix multiplication ($C = A \cdot A$) using custom nested loops versus the highly optimized CBLAS library, compiled with different optimization flags. The `time.h` library was used to measure operation times.

Optimization Flag	**Custom loops time [s]**	**CBLAS (OpenBLAS) time [s]**
None (-O0)	50	0
`-O1`	9	0
`-O2`	6	0
`-O3`	5	1
`-Ofast`	6	1

4 Conclusions

The project successfully demonstrated the implementation and performance evaluation of fundamental numerical linear algebra operations in C++. Depending on the chosen parameter ($\gamma = 2$ or $\gamma = 3$), the Euclidean norm of the resulting vector either increased or decreased. This behavior is connected to properties of generated matrix A.

Furthermore, the execution time comparisons emphasized the critical importance of usage of numerical libraries for heavy computational tasks. While compiling the naive nested loops with flags like `-O2` or `-Ofast` there can be seen noticeable speedups, the CBLAS library (`cblas_dgemm`) consistently provided the shortest execution times. This proves the efficiency of a CBLAS library and its functions.